The split we leave room for, and three others

Your scenario is correct. But the reasoning that makes it feel like the only honest split does not generalize, and an adversarial sweep of the code found three more honest-node splits distinct from yours. None are fixed by the hash tiebreak we just shipped. All of the fixes are prevention-class (like 4a); none reorder certified blocks or weaken finality, so none are the 4b you rejected.

1. Your scenario is correct, and it sharpens to one point

Two distinct same-height blocks each reaching quorum forces their signer sets to overlap. For a committee of N with quorum Q, the overlap is at least 2Q - N. With Q = 51 and N = 100 that is at least 2, so a double-certified split needs at least two nodes that signed *both* blocks. An honest node signs a second block at a height it already signed only if it no longer knows it signed the first: exactly your step 4 (a node that certified A1 but no longer has it). So your scenario reduces to honest equivocation via memory loss.

After the fixes we already made (countersignatures reload from disk on restart, 4a, and reorg-of-reorg re-seeding of anchor-invalidated blocks), a node that certified A1 keeps a durable record of it across restart and across the Bitcoin reorg, so it will not sign A2. The one residual trigger is a crash in the narrow window between emitting the signature for A1 and persisting anything. Closing it is a durable, write-ahead per-height signing record: before broadcasting a signature at height H, persist "I have committed to H = A1 anchored to X"; refuse to sign a different block at H while X is canonical, surviving crash, restart, and anchor-reorg. That is prevention, like 4a. It is not the reconciliation rule (4b) you rejected: it never reorders a certified block, it stops the second certificate from ever forming.

2. The arithmetic that makes amnesia feel like "the only way" is not general

The 2Q - N >= 2 overlap assumes the eligible signer set is exactly 100. It is not. Committee membership is threshold VRF sortition: a staker is in the committee for a slot iff `PosVrfSlot(...) < g_pos_committee_size` (pos.cpp:364). The *expected* size is 100, but the actual eligible count is a random variable, and the quorum is fixed at 51, computed from the nominal 100 (`PosQuorum(g_pos_committee_size)`, pos_producer.cpp:263), with no rule tying it to the actual eligible count.

Once the staker pool exceeds the committee target, which is the entire point of decentralized sortition, the eligible set exceeds 100 (roughly 40% of heights draw >= 102). A network partition that splits an eligible set of >= 102 into two halves of >= 51 gives two *disjoint* honest quorums, each certifying a different block at the same height, with no node ever signing twice. Safety needs 2Q > (actual eligible), but the code only guarantees 2Q > (expected eligible) = 100. This is a real honest-node double-certification with no equivocation at all, so it is not caught by the durable signing record.

This does not trigger on the current testnet, where the staker pool equals the committee target (100 fixed stakers), so every staker's slot is below 100 and the eligible set is always exactly 100. It is latent for the decentralized mainnet. Fix options: derive the quorum from the actual eligible count for the slot; or cap the certifying set to the top 100 by VRF rank so the actual size never exceeds 100; or over-provision the gap between expected committee size and the quorum threshold so that two disjoint honest quorums are cryptographically negligible (the Algorand-style margin).

3. Two more honest-node splits, distinct from yours

(a) Asymmetric-finality partition

A leader's block A1 at H+1 reaches quorum, one node assembles it, but the assembled block reaches only one side of a partition (or its flood is lost). That side finalizes A1. The other side never saw it; its finality floor stays at H, so escaping-stall lets it mint a rival at H+1 after the Bitcoin anchor gap. The A1 share-contributors stranded on that side have no durable record that A1 certified (they only emitted a share), so they honestly adopt the rival and sign the next height on top of it, and the rival chain reaches full quorum one height up. Since work equals height here, that chain then outgrows the A side. On heal, the A side is floor-pinned to A1 and rejects the taller rival chain as forking below its finalized block; the other side rejects A. Permanent, no node ever signed twice, and the hash tiebreak never runs because the two tips are at unequal work (and even at equal height each side stays floor-pinned).

Root cause: the finality floor is node-local and derived from the active chain only, and escaping-stall lets a partition mint a rival at a height where a certificate exists elsewhere but never crossed the partition. A certified block is orphaned by an honest supermajority: a safety break, not just a stall. Fix: gossip the certificate itself as a partition-crossing finality signal that pins all nodes, and suppress escaping-stall at or below any height where a certified sibling is known. A durable signing record alone does not fully close this; it caps the rival side below quorum but the floor-pinned side still cannot adopt the taller chain, so it degrades the safety split into an unhealable liveness split.

(b) Round-advance re-vote

The round index is a pure function of wall-clock time since the round-0 leader's block time (pos_producer.cpp DriveRound). If the round-0 leader L1 gathers its quorum near the round boundary but its assembled block propagates slower than one round, the nodes that have not yet seen L1 advance to round 1 (by the clock) and re-sign the next leader L2 at the same height, while their round-0 shares for L1 are still live and assemblable. If both cohorts reach 51, both L1 and L2 certify. The re-voting signers signed both, so the two double-signers the arithmetic needs arise from protocol-induced re-voting, not memory loss and not malice. This is a second honest double-signing channel, which is why the "only via amnesia" framing in section 1 is not the whole story.

Fix: a round-change lock rule (do not back or sign a round r > 0 rival at H while an earlier-round block at H is still potentially certifiable, analogous to 4a's "do not treat the height as vacant" hold but keyed on the round-advance race; or a lock/view-change certificate proving the earlier round cannot commit).

A blanket one-signature-per-height record would break the liveness that the re-vote exists to provide, so this one needs the round-aware version, not the amnesia fix.

4. Net

- The deployed hash tiebreak closes the symmetric sub-quorum *liveness* stall (neither side ever certifies). It does not touch any double-*certified* split, because both siblings are final and the tiebreak only orders sub-final blocks.
- A durable write-ahead per-height signing record closes your amnesia channel and the crash window. It does not close 3(a), 3(b), or section 2.
- Remaining, each a prevention-class fix (none reorder certified blocks, none weaken finality; none are 4b):
    - Section 2, disjoint quorums under committee-size variance: quorum from the actual eligible count, or a top-100 VRF-rank cap, or a wider committee-to-quorum margin.
    - 3(a), asymmetric-finality partition: certificate gossip as a finality signal, plus escaping-stall suppression where a certified sibling is known.
    - 3(b), round-advance re-vote: a round-change lock rule.

Confidence: section 2 is high (the mechanics are airtight; it is only latent because today's testnet has a fixed 100-staker pool). 3(a) and 3(b) are medium: the mechanisms are real and currently unguarded, but each needs a specific partition or timing race that I could not exercise on the live committee, so they are worth your judgment before we treat them as settled.

These came out of an adversarial code sweep (several independent passes, each finding candidates, then each candidate re-checked against the code and most discarded as duplicates of your scenario, the known sub-quorum stall, or already prevented). I did not implement any of the three fixes; they are consensus-level and yours to weigh, same as 4b.